
SEC & EDINET Metrics Mapping & Reconciliation

From raw filed XBRL to a standardized, reconciled fundamentals layer

The fundamentals AI Analyst reads are not the numbers companies file. They are the output of a multi-stage pipeline that ingests raw XBRL facts from SEC EDGAR (US) and EDINET (Japan), *maps* tens of thousands of heterogeneous filer concepts onto a single governed set of standardized line items, *derives* the items filers omit, *checks* them against accounting identities, *computes* a library of financial metrics, and *reconciles* every computed figure back to the exact source concepts and filings it came from.

This volume specifies that pipeline end to end: the raw-to-standardized concept mapping and its governance model, the graph-closure derivation and accounting-identity checks, the metric formula registry and its evaluation, and the reconciliation and validation layers that make every number auditable.

Companion volumes: System Architecture & Operations, Quantitative & ML Models, and qlib Integration.

Contents

Part I The standardized layer	2
1 Why a standardized layer exists	2
2 Pipeline overview	2
Part II Raw acquisition and parsing	3
3 US: SEC companyfacts	3
4 Japan: EDINET XBRL	3
Part III The standardization layer	3
5 The governed mapping model	3
6 Mapping resolution	4
7 Sign, unit, and value canonicalization	5
8 Graph closure: deriving what filers omit	5
8.1 Top-down residuals (catch-all children)	5
8.2 Bottom-up subtotals	5
8.3 Formula line items (L1)	5
9 Accounting-identity checks	6
Part IV Metrics computation	6
10 The formula registry	6
10.1 Dependency ordering and safe evaluation	7
10.2 Point-in-time context and period joins	7
11 International equities	7
Part V Reconciliation and validation	7
12 The reconciliation layer	7
12.1 Trace quality tiers	8
13 Validation and quality gates	8
14 How the standardized layer feeds the application	9

Part I The standardized layer

1 Why a standardized layer exists

Two companies reporting the identical economic fact — say, cost of goods sold — will tag it with different XBRL concepts, under different taxonomies, with different signs, scales, and statement positions, and one of them may not tag a subtotal the other reports explicitly. A US filer uses `us-gaap` concepts; a Japanese filer uses the EDINET/`jppfs` taxonomy in Japanese. Neither is directly comparable, and neither is stable across years as taxonomies evolve.

The `xbrl_sec` warehouse solves this with a *standardized layer*: a single, governed vocabulary of standardized line items (`revenue`, `cost_of_goods_sold`, `total_financial_debt`, ...) onto which every filer's raw concepts are mapped, and against which a uniform library of metrics is computed. AI Analyst reads only this layer; the application never touches raw filings. This document specifies how the layer is built and made auditable.

The invariant

Every standardized value and every computed metric can be traced back to the exact raw concept, filing, and fiscal period it derives from — or is explicitly marked as *derived* (from an accounting identity or a formula) rather than filed. The reconciliation layer (§12) records that trace for every figure, and the validation layer (§13) audits coverage and integrity.

2 Pipeline overview

Per jurisdiction the pipeline runs as an ordered set of stages. The application is read-only; these stages are the write path that maintains the shared warehouse.

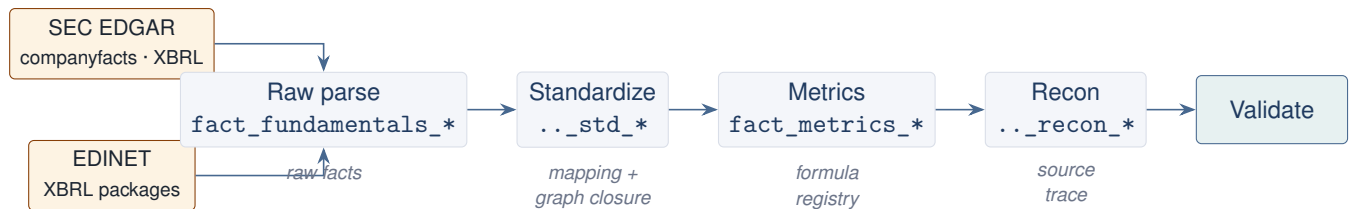


Figure 1. The end-to-end fundamentals pipeline. Each stage is idempotent and incrementally re-runnable; downstream stages have freshness gates so an unchanged input is skipped.

Stage	What it produces	Primary tables
Raw parse	Filed facts as (concept, value, unit, context, period, statement position)	<code>fact_fundamentals_{us,jp}</code>
Standardize	Standardized line items via governed mapping + graph closure	<code>fact_fundamentals_std_{us,jp}</code>
Metrics	Derived line items (L1) and financial metrics (L2) from the formula registry	<code>fact_metrics_{us,jp}</code>
Recon	Every metric re-expressed with its inputs and traced to source concepts/filings	<code>fact_metrics_recon_{us,jp}</code>
Validate	Coverage, identity, sparsity, and master-quality reports	(<code>report</code> ; <code>ref_std_identity_violation</code>)

The schema itself is defined as a large set of sequential, idempotent SQL migrations, each guarded so re-running the set against a current database is safe. The reference tables the standardizer depends on — the standardized line-item catalogue (`ref_standardized_line_items`), the rollup edges (`ref_std_item_edge`), the identity checks (`ref_std_identity_check`), and the governed concept

mappings — are loaded before any fundamentals run.

Part II Raw acquisition and parsing

3 US: SEC companyfacts

US fundamentals originate from SEC EDGAR’s structured *companyfacts* (and the XBRL filing packages for statement structure). Each raw fact carries its `us-gaap` (or filer-extension) concept, value, unit, the filing it came from, the fiscal period, and — from the calculation/presentation linkbases — its position in the statement tree (parent, root, weight, presentation order). These are written to `fact_fundamentals_us`.

Fiscal-period alignment. The SEC `fy` tag is the fiscal year of the *filing*, not of the reported period. A prior quarter that reappears as a comparative in a later 10-Q or 10-K inherits the later filing’s `fy`. Because the standardized key is (`cik`, `fiscal_year`, `fiscal_period`, `line_item`), two genuinely different quarters could otherwise collide. Annual rows are therefore re-dated from their `period_end`; quarterly rows are period-aligned to the fiscal year they close into, using the company’s inferred fiscal-year-end month — so a September-quarter fact for an early-FYE filer lands in the correct fiscal year rather than the filing’s.

4 Japan: EDINET XBRL

Japanese fundamentals originate from EDINET XBRL packages. The parser (`parsers/edinet_xbml.py`) reads the instance document together with the calculation and presentation linkbases:

- **Concept normalization** — an `href` fragment such as `jppfs_cor_Sales` is normalized to a `prefix/localname` concept id; non-numeric concepts (text blocks, abstracts, axes, domains, members, tables) are skipped.
- **Statement classification** — the linkbase `role` maps to a statement type (balance sheet / income statement / cash flow / equity) via role-string heuristics.
- **Context and dimensions** — each fact carries an XBRL *context* (period and any explicit dimensional members). JP standardization is context-aware: a `dimension_signature` distinguishes, for example, a consolidated total from a segment or a prior-period restatement, so the right context is chosen per line item.
- **Identity concepts** — cover-page DEI concepts (company name in English/Japanese, security code) are captured for entity resolution.

Facts are written to `fact_fundamentals_jp` with their context id, dimension signature, and statement structure, mirroring the US raw shape so the standardizer can treat both jurisdictions uniformly downstream.

Part III The standardization layer

5 The governed mapping model

The heart of standardization is a governed table, `map_concept_to_taxonomy_versioned`, that assigns each raw concept to a standardized `target_variable` under a set of qualifying conditions. Each mapping row carries:

Field	Meaning
target_variable	The standardized line item this concept feeds (e.g. <code>cost_of_goods_sold</code>). UNMAPPED rows are excluded.
tier	Mapping tier / precedence class (tier-1 primary vs. tier-2 secondary).
multiplier	Scale/sign multiplier applied to the raw value.
sign_policy / normal_balance	How the concept's sign is interpreted against the standardized item's normal balance.
mapping_sector	The sector scope the mapping applies to (<code>corp</code> , <code>bank_financial</code> , <code>insurance</code> , <code>reit</code> , ..., or <code>universal</code>).
gics_*	Optional GICS sector / industry-group / industry / sub-industry constraints.
effective_from/to_year	The fiscal-year range over which the mapping is valid (taxonomy evolution).
accounting_standard	Required accounting standard (e.g. US-GAAP, IFRS, JGAAP), or unconstrained.
taxonomy_version	Required taxonomy version, or unconstrained (year-level partial match allowed).
aggregation_type / priority	How multiple concepts mapping to the same target combine.
confidence	Governance confidence attached to the mapping.

Two sources feed the resolver: the governed versioned table above, and approved *company-period exceptions* (`map_concept_to_taxonomy_exception`, `review_status='approved'`, unexpired) that override the general mapping for a specific entity and fiscal-year range. Both are indexed by concept id with slash/colon aliasing so `prefix/local` and `prefix:local` forms both resolve.

6 Mapping resolution

For each raw fact, the resolver (`versioned_mapping.select_versioned_mapping`) selects the single best mapping by a deterministic specificity order. An approved exception matching the entity, year, period, sector, accounting standard, and taxonomy wins outright. Otherwise, among mappings valid for the fact's fiscal year:

1. **Accounting-standard & taxonomy gate.** A mapping is discarded if its required accounting standard or taxonomy version is incompatible with the fact. Compatible matches are scored (exact standard/taxonomy > same-year taxonomy > unconstrained), and these scores enter the ranking.
2. **Sector specificity.** Candidates are considered in the entity's *sector order* — most specific first, then universal. Corporate filers fall back `corp` → universal; banks, insurers, REITs, and asset managers use their own scopes plus universal (an insurer, for instance, resolves `insurance` → `non_bank_financial` → universal). The first sector with any qualifying candidate wins.
3. **GICS specificity.** Within that sector, a mapping's GICS constraints must match the entity; more specific GICS constraints (sub-industry > industry > group > sector) rank higher.
4. **Tie-breaks.** Remaining ties break by accounting/taxonomy score, then jurisdiction match, then earliest effective year, then confidence, then mapping id.

Why sector scope matters

Sector scope prevents category errors that silently corrupt derived figures. An insurer's operating investment portfolio, for example, must map to `investment_securities`, not to `short_term_investments` — otherwise it would be netted out as “excess cash” in corporate net-debt derivation. The resolver's sector hierarchy, plus sector-specific rollup edges (§8), keep each filer's economics on the right standardized items.

7 Sign, unit, and value canonicalization

Once a mapping is chosen, the raw value is canonicalized to the standardized item’s convention:

$$v_{\text{std}} = \begin{cases} -|v_{\text{raw}} \cdot \text{multiplier}|, & \text{line item is canonically negative,} \\ v_{\text{raw}} \cdot \text{multiplier}, & \text{otherwise.} \end{cases} \quad (1)$$

Canonically negative items — cost of goods sold, R&D, SG&A, interest expense, tax provision, capex, dividends paid, share repurchases — are stored with a consistent negative sign regardless of how the filer signed them, so downstream formulas can add line items without sign guesswork. Unit handling ranks candidate facts so a monetary line item prefers a currency-unit fact over a share-count fact of the same concept. The standardized row also records the source concept id, filing id, concept path, standardized concept path, mapping id, currency, and (for JP) the context id and dimension signature — the raw material for the recon trace.

8 Graph closure: deriving what filers omit

Filers report different subsets of a statement. After the raw mapping pass populates the *filed* leaves, `std/graph_closure.py` walks the standardized-item hierarchy (`ref_std_item_edge`, filtered to the entity’s sector scope and accounting standard) to derive missing values at the (entity, fiscal_year, fiscal_period) grain. It runs three passes.

8.1 Top-down residuals (catch-all children)

When a parent total is known and exactly one child is missing — preferentially a residual/“catch-all” child (item class `catch_all`, derivation policy `residual`, or an `other_*` item) — that child is derived as the parent minus its known siblings:

$$\text{child} = \pm \left(\text{parent} - \sum_{\text{known siblings}} \text{sign} \cdot \text{child} \right), \quad (2)$$

with the edge sign applied. This pre-pass runs first so an authoritative parent total fills a residual before a partial bottom-up subtotal can.

8.2 Bottom-up subtotals

Intermediate totals are then computed from their children. A parent is marked `DERIVED_BOTTOM_UP` when *all* children are known, or `DERIVED_PARTIAL` when at least 80% are known (below that it is left missing rather than fabricated from too little). Expense signs are handled once: a subtractive edge uses the magnitude of the child and applies the edge sign, so already-negative expense facts are not double-counted.

8.3 Formula line items (L1)

Registry-defined derived line items — the ones that are *always* computed rather than filed — are then evaluated from known values in dependency order (§10); for example

$$\text{total_financial_debt} = \text{short_term_debt} + \text{LTD_current} + \text{long_term_debt}, \quad (3)$$

$$\text{net_debt} = \text{total_financial_debt} - \text{cash} - \text{short_term_investments}, \quad (4)$$

$$\text{gross_profit} = \text{revenue} - |\text{cost_of_goods_sold}|. \quad (5)$$

Sector overrides apply here too — for insurers, `net_debt` drops the short-term-investment offset. Derived items carry a `metric_type` tag (`RESIDUAL`, `DERIVED_BOTTOM_UP`, `DERIVED_PARTIAL`, `DERIVED_TOP_DOWN`) so their provenance is never confused with a filed value.

9 Accounting-identity checks

Closure also *audits* the statements against accounting identities and emits violations. Two families of check run at the same entity×period grain.

Rollup consistency. For every parent whose children are all filed, the filed parent is compared to the signed sum of its filed children:

$$\left| \text{parent} - \sum_{\text{children}} \text{sign} \cdot \text{child} \right| > \tau \cdot |\text{parent}| \Rightarrow \text{violation}, \quad (6)$$

with a tolerance of $\tau = 5$ basis points of the parent magnitude.

Named identities. Declared checks in `ref_std_identity_check` (each a left-hand item, a set of right-hand items with signs, a tolerance, and a sector/standard scope) verify relations such as `Assets = Liabilities + Equity`. When all operands resolve, the signed residual is compared to the check's `tolerance_bp`. Failures are written to `ref_std_identity_violation` with the LHS value, RHS value, absolute delta, and delta in basis points — an auditable record of where a filer's statements do not tie out.

Advisory, not blocking

Identity violations are recorded for governance and surfaced to the committee's data-quality report, but they do not by themselves discard data: real filings occasionally fail an identity for benign reasons (rounding, disclosure differences). The application's committee treats them as advisory by default (see *System Architecture & Operations*); the warehouse keeps the evidence.

Part IV Metrics computation

10 The formula registry

Code: `metrics/formulas.py`, `metrics/compute.py`. Metrics are defined declaratively as executable formulas over standardized line items, generated from a line-item/metric registry so the public ids are governed in one place. There are two derived layers:

- **L1 line items** (`_L1_FORMULAS`) — derived *balance-sheet / income / cash-flow* quantities that behave like line items (e.g. `free_cash_flow`, `invested_capital`, `net_operating_assets`, `enterprise_value`, `tangible_book_equity`).
- **L2 metrics** (`_METRIC_FORMULAS`) — ratios, margins, growth rates, turnovers, coverage, yields, and composite signals (e.g. `return_on_invested_capital`, `free_cash_flow_yield`, `price_to_earnings_trailing`, `piotroski_f_score`, `sloan_accruals_ratio`).

Item / metric	Formula (registry)
<code>free_cash_flow</code>	<code>cash_flow_from_operations - capital_expenditures </code>
<code>invested_capital</code>	<code>total_equity + net_debt</code>
<code>net_operating_profit_after_tax</code>	<code>earnings_before_interest_taxes × (1 - cash_tax_rate)</code>
<code>return_on_invested_capital</code>	<code>nopat / invested_capital</code>
<code>gross_margin</code>	<code>gross_profit / revenue</code>
<code>free_cash_flow_yield</code>	<code>free_cash_flow / market_capitalization</code>
<code>price_to_earnings_trailing</code>	<code>market_cap / net_income_attributable_to_common</code>
<code>days_sales_outstanding</code>	<code>accounts_receivable_net × 365 / revenue</code>

10.1 Dependency ordering and safe evaluation

Formulas reference other derived items (`return_on_invested_capital` needs `nopat` and `invested_capital`, which need `cash_tax_rate` and `net_debt`). The registry is compiled into a directed dependency graph and *topologically ordered*; a cycle is a hard error. Each formula is evaluated in a sandboxed namespace with a fixed set of safe helpers — `_div` (null- and zero-safe division), `pct_change`, `cagr`, `_tax_rate`, `_abs_capex` — and no builtins, with a missing operand resolving to `None` so a partial input yields `None` rather than an exception. A result that is NaN or infinite is dropped.

10.2 Point-in-time context and period joins

Ratios that mix flows with market data or prior-period values are assembled with care. Market capitalization is computed point-in-time as $\text{price}(\text{period_end}) \times \text{shares}$; year-over-year and multi-year formulas join the correct prior annual period (`prev`, `3y`, `5y`) via `*_prev / *_3y / *_5y` operands resolved against the entity’s own annual history. A handful of “special” metrics (momentum, volatility) come from the price/market path rather than the fundamentals formulas. Each metric row stores its `formula`, `value`, `currency`, `metric_type`, `category`, `importance`, `unit_type`, and a `fallback_applied` flag.

11 International equities

Non-US/JP equities follow a parallel, lighter path (`metrics/compute_intl.py`) that computes the same metric ids from Yahoo-sourced fundamentals (`fact_yahoo_*`) into `fact_metrics_intl`. It reuses the formula registry where the inputs are available and degrades where they are not, so the screener and quant panels can include international names on the same metric vocabulary.

Part V Reconciliation and validation

12 The reconciliation layer

Code: `metrics/recon.py` → `fact_metrics_recon_{us,jp}`. Reconciliation re-expresses every computed metric with its actual inputs and traces each input back to the raw source. For a metric row it records:

Column	Content
<code>formula</code>	The symbolic formula, e.g. <code>_div(free_cash_flow, market_capitalization)</code> .
<code>formula_with_values</code>	The same formula with each operand substituted by its value and name, e.g. <code>_div(1.84e10 [free_cash_flow], 2.7e12 [market_capitalization])</code> .
<code>input_values</code>	The resolved numeric value of every operand (<code>None</code> where missing).
<code>source_line_items</code>	The standardized line items the operands resolved to.
<code>source_concept_ids</code>	The raw XBRL concepts behind those line items.
<code>source_filing_ids</code>	The filings those facts came from.
<code>raw_trace</code>	Per-operand records: line item, value, source concept, filing, form, filed date, concept path, period role (<code>current</code> / <code>prev</code> / <code>3y</code> / <code>5y</code>), and (JP) context.
<code>trace_quality</code>	A tier describing how completely the metric traces to source (below).

12.1 Trace quality tiers

Each reconciled metric is graded by how fully its inputs trace to filed source:

trace_quality	Meaning
full_source_trace	Every operand resolves to a filed source fact.
partial_computed_inputs	Some operands are themselves derived (L1) rather than filed; values are present.
partial_missing_inputs	One or more operands are missing (the metric may still compute from the rest).
computed_only	Inputs exist as values but none resolve to a source fact (fully derived).

To build the trace, recon reconstructs the same L1 line-item time series the metrics stage used, resolves each formula token to its period-appropriate source (current vs. prev/3y/5y), and de-duplicates the resulting source records. The build is entity-chunked to bound memory, and has a freshness gate so it is skipped when the recon table already covers the metrics table.

Worked example — free cash flow yield

For one entity-period, `free_cash_flow_yield=_div(free_cash_flow, market_capitalization)`. Recon records:

- `formula_with_values:` `_div(1.84e10 [free_cash_flow], 2.70e12 [market_capitalization])`
- `free_cash_flow` is itself L1 (`cash_flow_from_operations - |capital_expenditures|`); its operands trace to the filed `us-gaap:NetCashProvidedByUsedInOperatingActivities` and `us-gaap:PaymentsToAcquirePropertyPlantAndEquipment` concepts in a specific 10-K filing.
- `market_capitalization` is computed point-in-time from `price × shares`.
- `trace_quality: partial_computed_inputs` — the fundamentals trace to source, but market cap is a derived `price×shares` quantity, not a filed fact.

Every figure the committee cites can thus be unfolded to the filings and concepts underneath it.

13 Validation and quality gates

Code: `quality/validate.py`. A compact validation summary is produced per jurisdiction, spanning the whole pipeline:

- **Layer counts** — rows, entities, and distinct concepts/line-items/metrics at each of the raw, standardized, metric, and recon layers, so shrinkage or gaps between layers are visible at a glance.
- **Master quality** — hard gates on the company master (`dim_company_{us,jp}`): e.g. US excludes ETFs/ETNs, funds/trusts, and non-operating SIC codes; JP enforces primary-ticker format, ISIN format, security-code/ticker consistency, and ticker-link integrity. A failed gate raises rather than silently shipping bad master data.
- **Source and XBRL state** — per-source download/extract/parse progress and parse-error counts from `source_filing_state`, including US XBRL linkbase (`cal/pre/def/lab`) acquisition status.
- **Structural sparsity** — coverage of the structural columns (statement type, parent/root, presentation order, weights) that graph closure depends on.
- **Recent runs** — the last pipeline stage runs with row-in/row-out counts and errors, from `pipeline_stage_run`.

14 How the standardized layer feeds the application

The application reads only downstream of this pipeline. The committee’s evidence engine reads standardized fundamentals and metrics; the quant desk builds its panel from `fact_metrics_*`; the WACC and factor models read the factor loadings derived from prices. The committee also ships a read-only *data-quality agent* that scans exactly the five layers this document describes — raw, standardized, metrics, recon, and a cross-check against an independent source — reports layer scores and severity-ranked findings, and (in the mapping-check path) surfaces raw concepts for an entity that went unmapped or resolved to an incompatible sector, routing any proposed mapping change to a review queue rather than mutating the governed table. The reconciliation trace of §12 is what lets that agent, and any analyst, unfold a headline number to the filings beneath it.

The standardized layer is the contract between heterogeneous filings and a single, auditable analytical vocabulary. Every number the application shows can be traced to the filing it came from, or is explicitly marked as derived.